

Support de Presentation : QR Attendance

Architecture de Deploiement, Structure du Projet & Jalons Fonctionnels

1. Architecture & Strategie de Deploiement

Nous avons adopte une architecture cloud moderne, decouplee et conteneurisee pour assurer la flexibilite, la securite et la gestion optimale des couts de l'application :

- **Base de données (Neon.tech)** : Serveur PostgreSQL manage serverless. Neon permet la mise en veille automatique des ressources de calcul lorsque la base de donnees n'est pas sollicitee (scale-to-zero), optimisant ainsi les couts d'hebergement.
- **Service Backend (Render - Docker)** : Le backend Django REST est package sous forme de conteneur Docker. Cela garantit un comportement identique entre l'environnement de developpement et de production. Il est deploye sur un Web Service Render et execute via Daphne (serveur ASGI) pour supporter des connexions HTTP/WebSocket concurrentes et fluides (essentiel pour la synchronisation en temps reel du scan de presence).
- **Service Frontend (Render - CDN)** : L'application React est compilee en fichiers statiques optimises (HTML/JS/CSS) et hebergee sur le CDN mondial de Render. Nous avons configure une regle de reecriture (SPA rewrite rule : /* -> /index.html) pour permettre aux routes du client React-Router d'etre rechargees directement dans le navigateur sans renvoyer d'erreur 404.

2. Configuration des Variables d'Environnement en Production

Pour des raisons de securite, toutes les configurations sensibles sont injectees via des variables d'environnement sur le tableau de bord de Render :

- **DATABASE_URL** : Chaine de connexion securisee vers la base de donnees PostgreSQL de Neon.
- **DJANGO_DEBUG** : Definie sur False en production pour masquer les rapports d'erreurs detailles.
- **CORS_ALLOWED_ORIGINS** : Limite la consommation de l'API backend au seul domaine de l'application frontend.
- **CSRF_TRUSTED_ORIGINS** : Autorise le domaine frontend a soumettre des formulaires d'edition ou de creation securises.
- **FRONTEND_BASE_URL** : Domaine de l'application frontend utilise par le backend pour generer dynamiquement l'URL cible encodee dans le QR code.
- **VITE_API_URL** : URL du backend Django REST consume par le client React pour executer ses requetes API.

3. Structure Globale du Projet

Le projet est decoupe en deux grandes structures independantes :

- backend/ : Service Django gerant les modeles de donnees, l'authentification et l'API.
- frontend/ : Client React gerant l'affichage utilisateur, le scan et les tableaux de bord administratifs.

```
qr-attendance/
+-- backend/                                # Service Backend Django
|   +-- Dockerfile                          # Configuration Docker production
|   +-- core/                              # Configuration principale Django (settings, urls, asgi)
|   +-- gestion_presence/                  # Application de presence
|       +-- models/                        # Modeles de donnees (User, Enseignant, Filiere, Module,
Cours)
|       +-- api/                           # Vues d'API (auth_views, qrcode_views, seance_views,
urls)
|
+-- frontend/                              # Application Frontend React
    +-- src/
        +-- components/                    # Composants partages (Sidebar, ProtectedRoute)
        +-- config.ts                      # Resolveur d'URL API
        +-- pages/                         # Pages de l'application (Dashboard, Scan, etc.)
            +-- EnseignantsPage             # Gestion des enseignants et matieres
            +-- EtudiantsPage               # Import Excel & Saisie manuelle etudiants
            +-- FilieresModulesPage         # Gestion des filieres & modules
            +-- ScanPage                    # Validation de presence par code Massar
```

4. Jalons Fonctionnels Realises

- Generation automatique des identifiants : Lors de la creation manuelle d'un enseignant ou d'un etudiant, l'application genere automatiquement son adresse e-mail professionnelle (prenom.nom@umi.ac.ma) et son mot de passe par default si ces champs sont laisses vides.

- Importation en masse par Excel/CSV : Les administrateurs peuvent importer une liste d'etudiants a partir de fichiers Excel (.xlsx) ou CSV. L'application valide l'absence de doublons de code Massar ou d'e-mail, insere les comptes valides et renvoie un rapport detaille en cas de lignes en erreur.

- Gestion des Filieres et Modules (Admin) : Mise en place d'une interface d'administration fluide. L'administrateur peut creer/modifier des filieres et y associer des modules par semestre. La selection d'une filiere filtre automatiquement les modules affiches.

- Association des Enseignants : Le modele Enseignant a ete enrichi d'une relation Many-to-Many avec les filieres et modules. L'admin peut assigner les matieres qu'enseigne chaque professeur, et les filieres apparaissent sous forme de badges dans la liste.

- Scan avec geolocalisation & correction Massar : Le scan de presence verifie la position geographique de l'etudiant par rapport a la salle de cours. De plus, le champ du Code Massar a ete configure pour accepter les caracteres alphanumeriques afin de permettre la saisie des prefixes de lettres (D, R, etc.) sur smartphone.

- Filtrage par Filiere et Semestre (Enseignant) : L'enseignant dispose maintenant de dropdowns de selection sur son emploi du temps pour filtrer les seances affichees par filiere et semestre, lui permettant de visualiser uniquement les cours correspondants.

- Changement de mot de passe (Parametres) : Les utilisateurs peuvent desormais modifier leur mot de passe directement depuis la page des parametres. L'application valide la correspondance et la complexite (min. 6 caracteres) du nouveau mot de passe cote client et cote serveur.